

A PROJECT REPORT ON

Pest Detection And Extraction Using Image Processing Techniques

A project report submitted in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Computer Applications

Submitted By

K.Poojitha (Regd.No. 2018-1902048)

N.Pavan Kumar Choudhury (Regd. No. 2018-1902060)

Aasim Ali Shaik (Regd. No. 2018-190205)

Under the Esteemed Guidance of

Smt. CH.SUNEETHA

Assistant Professor

Department of Computer Application



**GAYATRI VIDYA
DEGREE AND P.G. COURSES**

PARISHADCOLLEGE FOR

(AUTONOMOUS)

(Affiliated to Andhra University)

RUSHIKONDA, VISAKHAPATNAM.(2017-2020)

GAYATRI VIDYA PARISHAD
COLLEGE FOR DEGREE and P.G COURSES (Autonomous)
(Affiliated to Andhra University)
RUSHIKONDA, VISAKHAPATNAM
Department of Computer Application



CERTIFICATE

This is to certify that the project report entitled “**Pest Detection And Extraction Using Image Processing Techniques**” is the Bonafide record of project work carried out by **K.Poojitha (Regd. No.2018-1902048)**, **N.Pavan Kumar Choudhury (Regd.No.2018-1902060)** and **Aasim Ali Shaik (Regd. No.2018-1902055)**, are the students of this college, during the academic year **2018-2021**, in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications.

Project Guide

Smt. CH.SUNEETHA

Assistant Professor

Head of the Department

External Examiner

DECLARATION

We **K.Poojitha , N.Pavan Kumar Choudhury and Aasim Shaik Ali** hereby declare that the project report entitled “**Pest Detection and Extraction Using Image processing techniques**” is an original work done at **Gayatri Vidya Parishad College for Degree And P.G Courses (A), Visakhapatnam**, submitted in partial fulfillment of the requirements for the award of Bachelor of Computer Applications, to Gayatri Vidya Parishad College for Degree And P.G Courses (A), affiliated to Andhra University. We assure that this project is not submitted in any other University or college.

(K.Poojitha Prasanna)(2018-1902048)

(N.Pavan Kumar Choudhury)(2018-1902060)

(Aasim Ali Shaik)(2018-1902055)

ACKNOWLEDGEMENT

I consider it as a privilege to thank all those people who helped me a lot for successful completion of the project “**Pest Detection and Extraction Using image processing techniques.**”

First of all I would like to thank our beloved principal **Prof. S. Rajani of Gayatri Vidya Parishad College for Degree And P.G Courses(A)**, who has provided full- fledged lab and infrastructure for successful completion of my project work.

I would like to thank our ever-accommodating Head of the Department, Bachelor of Computer Applications Sri **P.Venkata Rao**, and my guide **Smt. CH.SUNEETHA, Assistant Professor** has obliged in responding to every request though they are busy with their hectic schedule of administration and teaching.

I thank all the **Teaching & Non-Teaching staff** who has been a constant source of support and encouragement during the study tenure.

(K.Poojitha Prasanna)(2018-1902048)

(N.Pavan Kumar Choudhury)(2018-1902060)

(Aasim Ali Shaik)(2018-1902055)

Pest Detection

And

Extraction

Using Image

Processing

Techniques

ABSTRACT

Detection of pests in the fields is a major challenge in the field of agriculture, therefore effective measures should be developed to fight the infestation while minimizing the use of pesticides. The techniques of image analysis are extensively applied to agricultural science and it provides maximum protection to crops which can ultimately lead to better crop management and production. Monitoring of pests infestation requires a huge amount of manpower, however automatic monitoring has been advancing in order to minimize human efforts and errors. This project extends the implementation of different image processing techniques to detect and extract insect pests by establishing an automated detection and extraction system for estimating pest densities in paddy fields.

CONTENT

Topic	Page no
--	
1. Introduction	9
2. Literature Survey	
2.1. Introduction	16
2.2. Current system	16
2.3 Proposed system	16
2.4 Objectives	17
2.5 Requirement Elicitation	17
2.5.1 Hardware requirements	17
2.5.2 Software Requirements	17
2.6 Example	18
3. UML Modeling	
3.1 Introduction	19
3.1.1 Actors	19
3.1.2 About Use –Case Diagram	20
3.2 About Sequence Diagram	22
3.2. 1 Sequence Diagram	22

4. Design

4.1. Design and description of algorithm 26

4.1.1 Introduction to design 26

4.1.2 Technical Criteria for design 26

4.1.3 Design Principles 26

4.2 Design and Algorithm

4.2.1 27

4.2.2 28

5. Coding

5.1 Coding Approach 30

5.2 Information Handling 30

5.3 Programming Style 31

5.4 Verification and Validation 31

5.5 Sample Source Code 31

6. Testing

6.1. Levels of Testing 66

6.2. Testing Approaches 67

6.3. Testing Activities 68

6.4. Testing Types 68

7. Result

8. Conclusion	74
9. References	
10. Appendix	
10.1 List of Figures	75

1. INTRODUCTION

Pest Detection And Extraction Using Image Processing Techniques

Rice is the most important and a primary source of food in Asia especially in the Philippines however, rice may lose its quantity and quality when being attacked by different insect pests. Therefore, it is a top priority to find effective methods to reduce the level of their infestation in the paddy fields. In agriculture, pest control has always been a challenging task for the farmers. Most of the farmers use regular pest management methods which is the regular spray program based on schedules rather than the presence of insect pests on the paddy fields. These chemicals will kill useful insects which eradicate pests in crops. Assessing the density of rice pest population in paddy fields is very important for pest forecasting decisions. Sticky traps are widely used to trap insect pests, the trapped insects are brought to the laboratory for counting and identification manually. Usually, crop technicians identify and segregate the insects manually according to their species and count the major pests separately. The resulting counts are used to estimate the pest density in the paddy fields. Due to the rapid development of digital technology, there is an opportunity for image processing technology to be used in the field of agricultural research which can solve a complex problem image analysis provides a realistic opportunity of insect pest detection.

Detection of pests in the fields is a major challenge in the field of agriculture, therefore effective measures should be developed to fight the infestation while minimizing the use of pesticides. The techniques of image analysis are extensively applied to agricultural science and it provides maximum protection to crops which can ultimately lead to better crop management and production. Monitoring of pests infestation requires a huge amount of manpower, however automatic monitoring has been advancing in order to minimize human efforts and errors. This project extends the implementation of different image processing techniques to detect and extract insect pests by establishing an automated detection and extraction system for estimating pest densities in paddy fields.

2.LITERATURE SURVEY

2.1 INTRODUCTION:

Rice is the most important and a primary source of food in Asia especially in the Philippines however, rice may lose its quantity and quality when being attacked by different insect pests. Therefore, it is a top priority to find effective methods to reduce the level of their infestation in the paddy fields. In agriculture, pest control has always been a challenging task for the farmers. Most of the farmers use regular pest management methods which is the regular spray program based on schedules rather than the presence of insect pests on the paddy fields. These chemicals will kill useful insects which eradicate pests in crops. Assessing the density of rice pest population in paddy fields is very important for pest forecasting decisions. Sticky traps are widely used to trap insect pests, the trapped insects are brought to the laboratory for counting and identification manually. Usually, crop technicians identify and segregate the insects manually according to their species and count the major pests separately. The resulting counts are used to estimate the pest density in the paddy fields. Due to the rapid development of digital technology, there is an opportunity for image processing technology to be used in the field of agricultural research which can solve a complex problem image analysis provides a realistic opportunity of insect pest detection.

2.2 CURRENT SYSTEM:

Sampling method : This method was used by farmers to achieve this goal because they used this to collect, count or inspect a small part of pest population.

Different Sampling methods :-

Light trap : This method involves varying size sample which is good for composing seasonal and yearly catches of insects, but catches are subject to insects.

Sweep method : this method refers to catching insects using fish nets, this is a fast method, very economical and good for sampling arthropods staying in canopy of rice. But it has human error due to variability and poor catch of arthropods at the base of the plant.

Tapping the rice : This method uses a collecting pan with soap solution or oil with water to collect arthropods at the base and stem of the rice. It is very costly method because of heavy labor cost.

Stick trap : This method measures insect movement and colonization and does not catch flying insects.

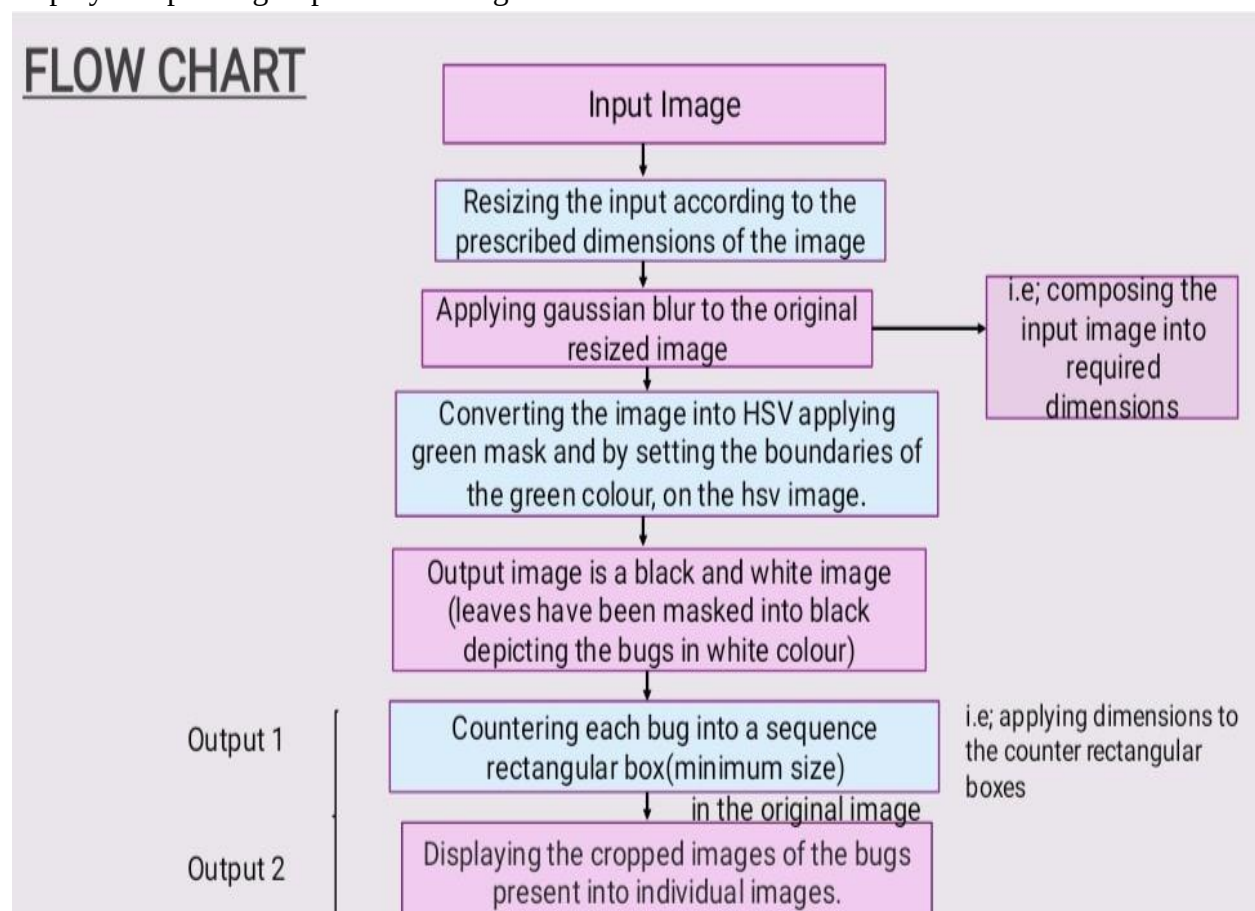
Yellow pan trap : This method measures insect migration, easy sorting and counting of samples. But the attraction is due to color stimulus and does not catch non flying insects

2.3 PROPOSED SYSTEM:

Image processing : Image processing is the analysis and manipulation of graphical images from sources such as photographs and videos.

There are three main steps in Image processing :-

Considering a captured image, converting the captured images into binary values that a computer can process. Image enhancement and data compression. This is an output step that consists of display and printing of processed image.



2.4 OBJECTIVES:

Integrated pest management regulates pests by using a variety of control measures, including mechanical, cultural, biological, and chemical. **Management rather than eradication of pests** is the goal. An IPM plan begins with a careful evaluation of each pest infestation

To manage insects by not only killing them but by preventing feeding, multiplication and dispersal. 3. To use ecofriendly methods, which will maintain quality of environment (air, water, wild life and plant life

2.5 Requirement Elicitation:

A Requirement is a feature that the system must have or a constraint that it must satisfy to be accepted by the clients. Requirements Engineering aims at defining the requirements of the system under construction.

2.5.1 Hardware Requirements

- RAM: 6 GB and above
- PROCESSOR: Intel(R) Core(TM) i3-8250U CPU @1.80GHz
- KeyBoard (standard 102 keys)
- Mouse(standard)

2.5.2 Software Requirements

- OPERATING SYSTEM: Windows 7 and above
- PACKAGE: Visual Studio Code
- PROGRAMMING LANGUAGE: Python Programming Language

3. UML MODELING

3.1 Introduction:

The **UML** is a tool for specifying software systems. Standardized diagram types to help you describe and visually map a software system's design and structure. Using **UML** it is possible to model just about any kind of application, both specifically and independently of a target platform. While **UML** is naturally oriented towards Object-Oriented programming, but it is just as easy to model procedural languages such as C, Visual Basic, Fortran etc.

The use of **UML** as a tool for defining the structure of a system is a very useful way to manage large, complex systems. Having a clearly visible structure makes it easy to introduce new people to an existing project.

Building up libraries of UML Patterns simplifies the re-use of models and code. UML Patterns are groups of collaborating objects/classes that can be abstracted from general modeling scenarios. As patterns are discovered in a new project, pattern templates can be re-used and modified by the designer to fit the new project.

3.1.1 Actors:

Actor: Actors are external entities that interact with the system. Actors typically include a user role or another system. They have unique names and descriptions

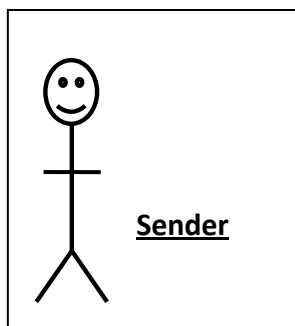


Fig: Sender actor

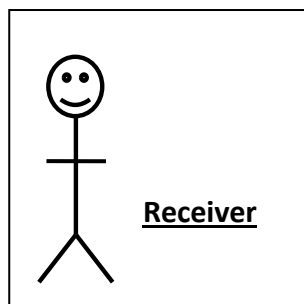


Fig: Receiver actor

3.1.2 About Use case diagram:

A **use case diagram** at its simplest is a representation of a user's interaction with the system and depicting the specifications of a use case. A use case diagram can portray the different types of users of a system and the various ways that they interact with the system. This type of diagram is typically used in conjunction with the textual use case and will often be accompanied by other types of diagrams as well.

Use Cases: Use cases describe the behavior of the system as seen from the actor's point of view. It describes the function provided by the system as a set of events that yield a visible result for the actors.

Actors initiate the use cases for accessing system's functionality. When actors and use cases exchange information, they are said to **Communicate**. To describe a use case we use a template composed of six fields:

Use Case Name: The name of the use case

Participating Actors: The actors participating in the particular use case

Entry Condition: Condition for initiating the use case.

Flow of events: Sequence of steps describing the functioning of use case.

Exit Condition: Condition for terminating the use case.

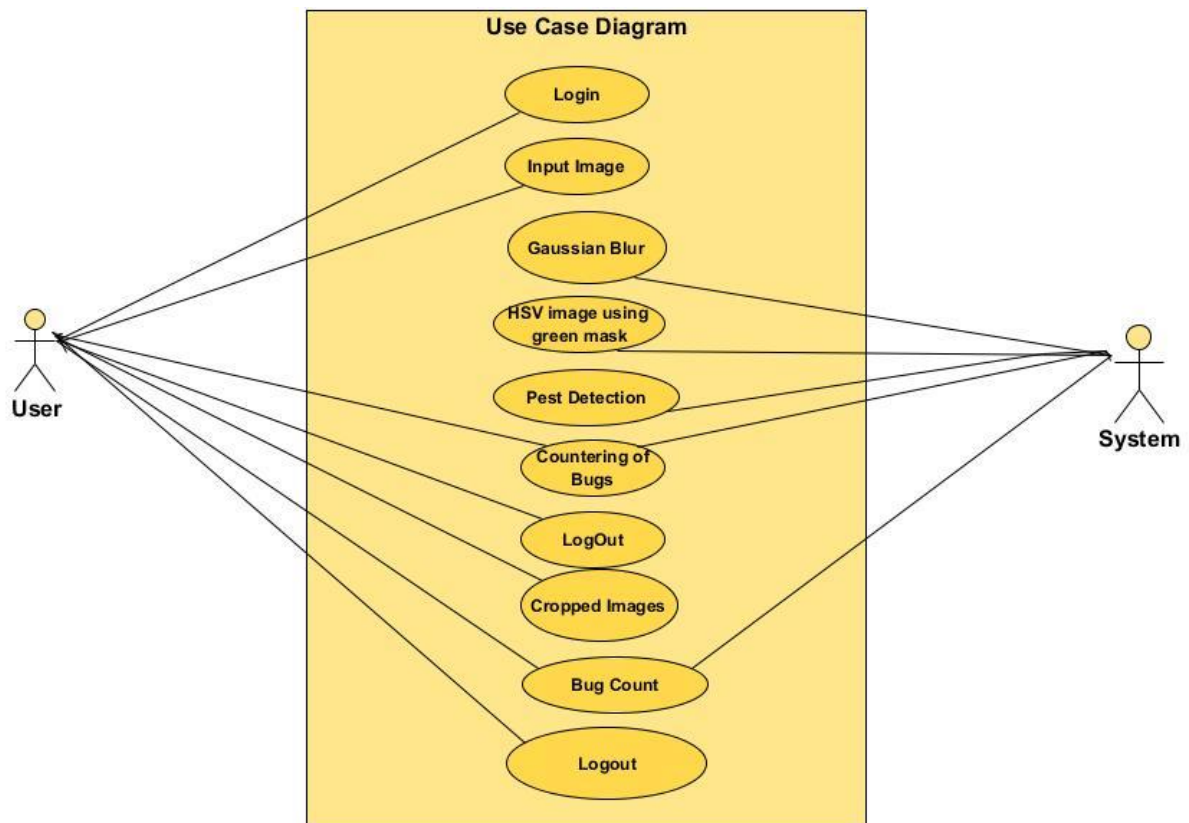
Quality Requirements: Requirements that do not belong to the use case but Constraint the functionality of the system.

Use case diagrams include four types of relations. They are as follows:

- Communication Relationships
- Inclusion Relationships
- Extension Relationships
- Inheritance Relationships

USE CASE DIAGRAM

(Fig 3.1.2)

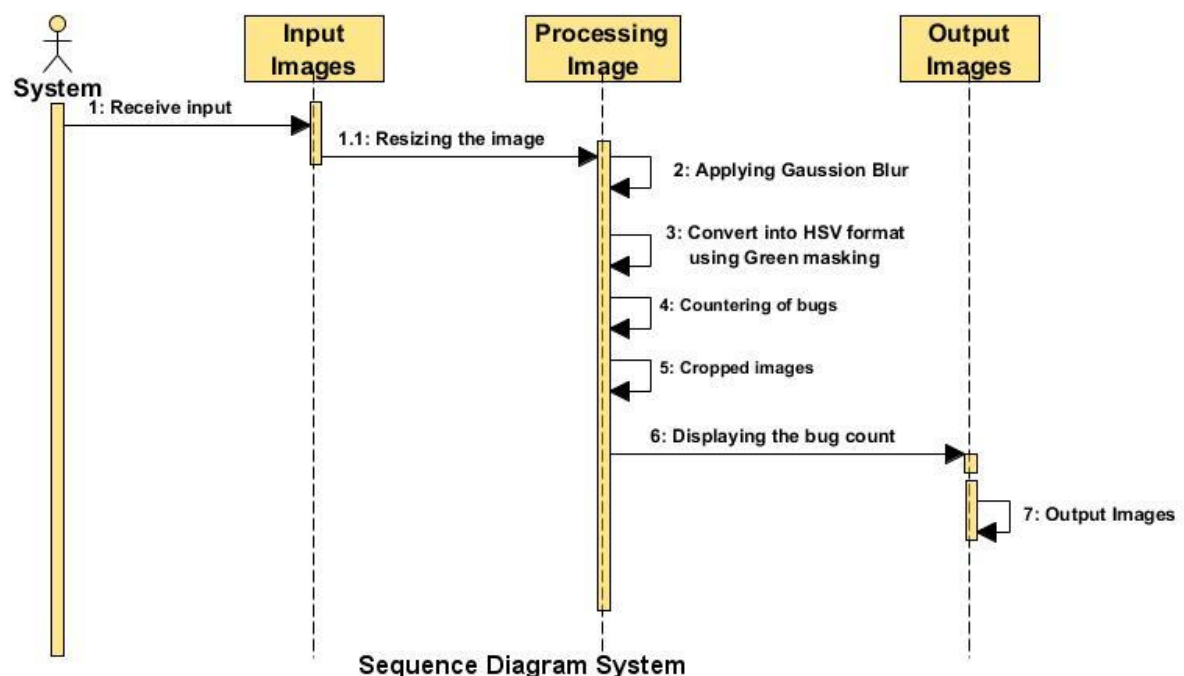
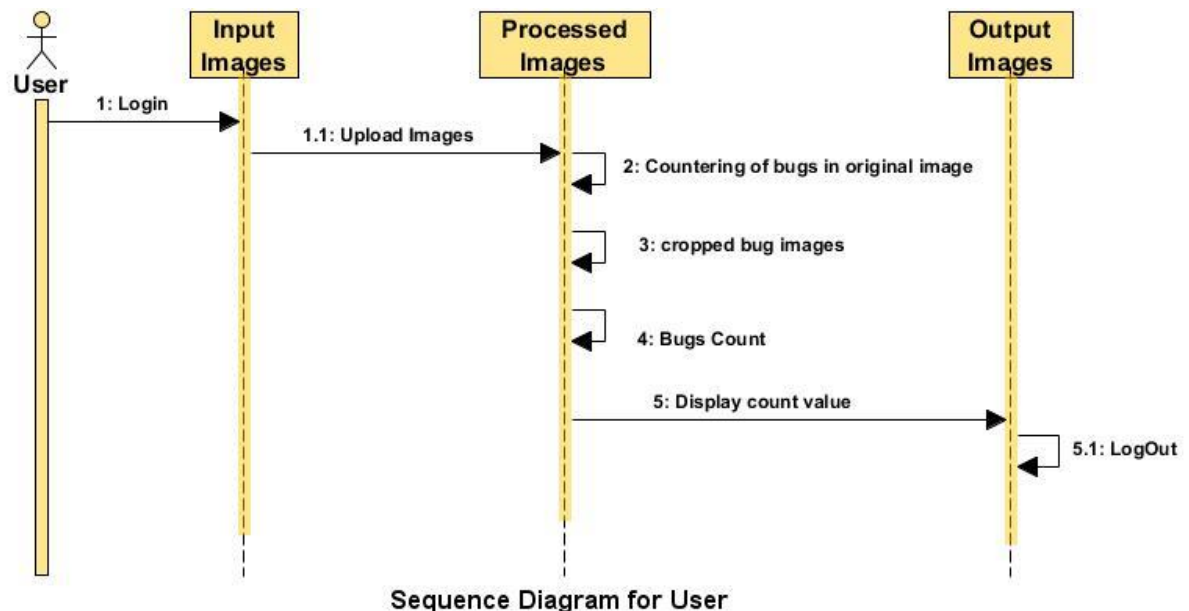


3.2 About Sequence Diagram:

Interaction between objects can be described by means of sequence diagrams. An object interacts with another object by sending messages. The reception of a message by an object triggers the execution of an operation, which in turn may send messages to other objects. Arguments may be passed along with a message and are bound to the parameters of the executing operation in the receiving object. Typically, we use a sequence diagram to the event flow of a use case, identify the objects that participate in the use case, and assign pieces of the use case behavior to the objects in the form of services.

3.2.1 Sequence Diagram

(Fig 3.2.1)

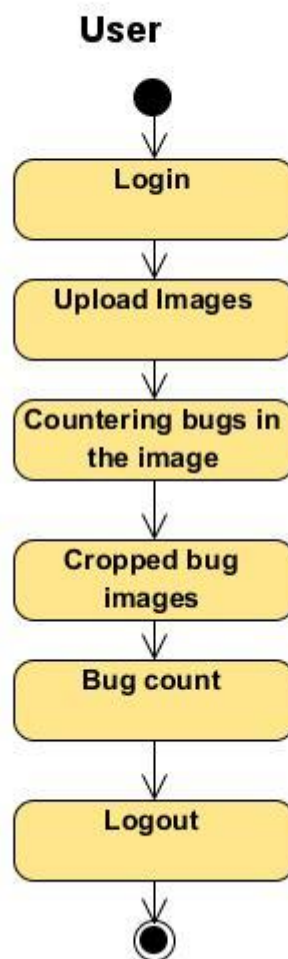


3.3 State-Chart Diagram:

State diagrams are used to describe the behavior of a system. State diagrams describe all of the possible states of an object as events occur. Each diagram usually represents objects of a single class and tracks the different states of its objects through the system. Not all classes will require a state diagram and state diagrams are not useful for describing the collaboration of all objects in a use case. State diagrams have very few elements. The basic elements are rounded boxes representing the state of the object and arrows indicating the transition to the next state. The activity section of the state symbol depicts what activities the object will be doing while it is in that state. All state diagrams begin with an initial state of the object. This is the state of the object when it is created. After the initial state the object begins changing states.

State Chart Diagram for USER

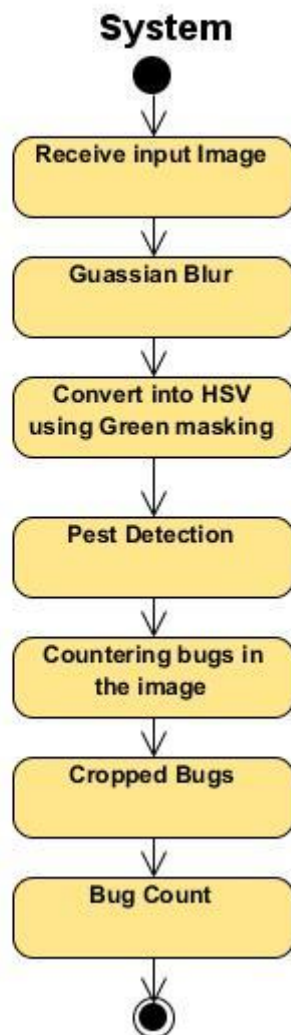
State Chart Diagram for User



(Fig 3.3.1)

State Chart Diagram for SYSTEM

State Chart Diagram for System



4. DESIGN

4.1 DESIGN AND DESCRIPTION OF ALGORITHM

4.1.1 Introduction to Design

Design is the first step in development phase. It is the first step into the solution domain. The analysis and specification deals with the problem domain. Analysis deals with what a proposed system has to do, where design deals with how the system implements. It can be defined as "the process of applying various techniques and principles for the purpose of defining a device, a process or a system in sufficient detail to permit its physical realization".

4.1.2 Technical criteria for design

A design should exhibit a hierarchical organization that makes intelligent use of control among element of software. A design should be modular that is the software should be logically partitioned into elements that perform specific functions and sub functions. A design should contain both data and procedural abstractions. A design should lead to modules that exhibit independent functional characteristics A design should lead to interface that reduce the complexity of connections between modules and with the external environment A design should be derived using a repeatable method that is driven by information obtained during the software requirements analysis.

4.1.3 Design Principles

Software design is both a process and a model. The design process is a set of iterative steps that enable the designer to describe all aspects of the software to be built. Basic design principles enable the software engineer to navigate the design process. Some of the Design Principles are

- The design should be traceable
- The design not reinvents the wheel
- The design should "minimize the intellectual distance between and the problem as it exists in the real world"
- The design should exhibit uniformity and integration

Design methodology is a systematic approach to create a design by applying a set of techniques and guidelines. Most design methodologies focus on the system design. Problem partitioning is a basic principle for guiding the design process. The goal is to effectively handle the complexity involved in designing large systems, by partitioning their problem into separate pieces so that each piece can be solved separately. Problem partitioning depends on the effective use of abstraction. Abstraction of a component is a view of the component that extracts the essential information relevant to particular focus and ignores the rest of the information. It allows a module to be specified by its external behaviour only. Abstraction permits concentrating on one component

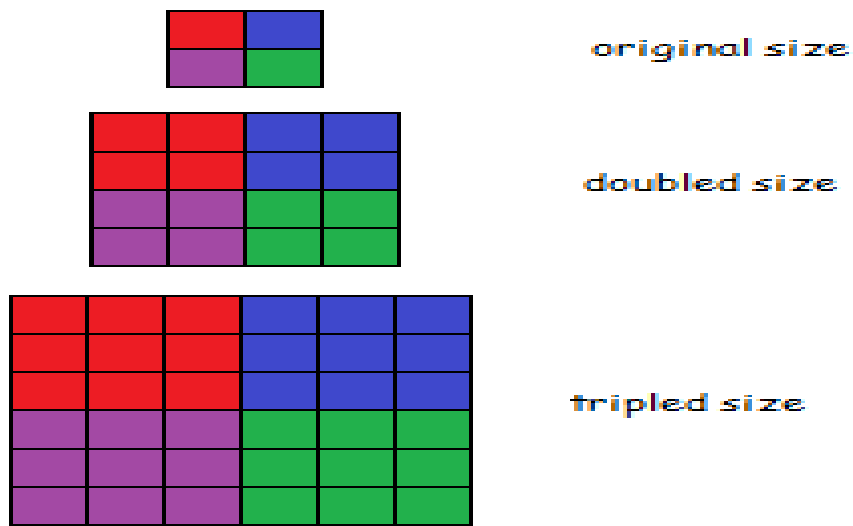
at a time by using the abstraction of other components to decide the interaction between the components. Design is the first step in the development phase for any system .it is the first of the three technical activities design; code generation and testing that are required to build and verify the software. The design step produces a data design, an architectural design an interface design and procedural design. The data design transforms the information domain model created during the analysis into the data structures that will be required to implement the software. The data objects and the relationships defined in the data dictionary provide the basis for the design activity. The architectural design defines the relationship among major structural elements of the program. The interface design describes how the software communicates with itself, to systems that interoperate with it, and humans who use it. The procedural design transforms the structural elements of the programs architecture into procedural description of the software components. Design is the only way that we accurately translate a customer's requirements into finished software product or system. Without design we risk in building an unstable system-one that will fail when small changes are made; one that may be difficult to test; one whose quality cannot be accessed until late in the software engineering process, then time is short and many dollars have been spent.

Pose animator is a free and open source project and it is all about driving a 2d vector graphic based application using a set of bones that is actually calculating from your webcam.

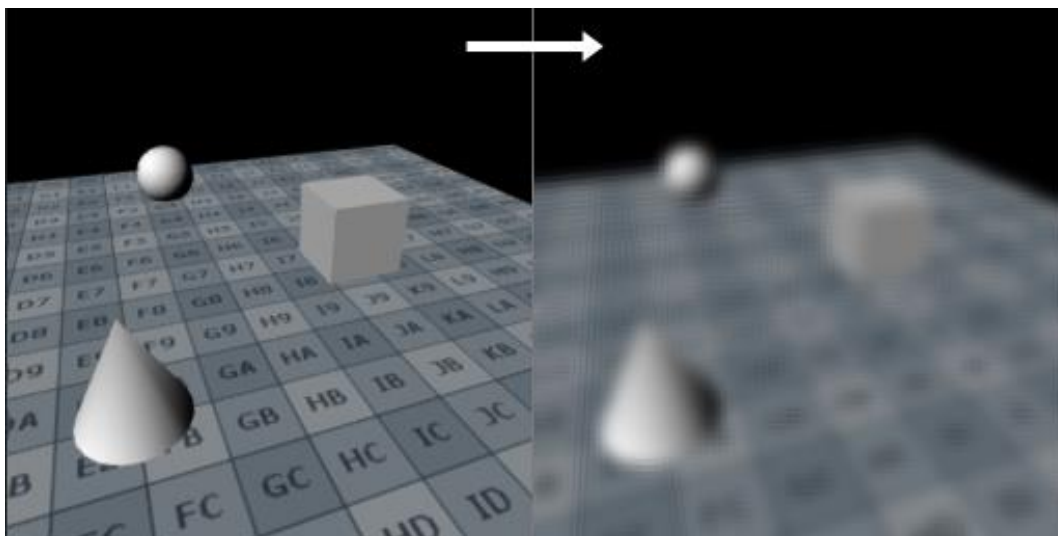
4.2 Design and Algorithm

4.2.1Input image: The user inputs an image in the system where the input image can be of any size and resolution and also can be with or without bugs/pests in image.Input image is the first and primary source for the initiation of the system to begin.Input image undergoes different processing techniques to depict the required data to the user

Resizing the image: In this step, the image being input by the user will be converted into default image size prescribed by the system for the upcoming process to be taken place. Only the resized image will be processed to the next step.Resizing of the image refers to conversion of image of any dimensions into default image dimensions set by the system inorder to process the image.

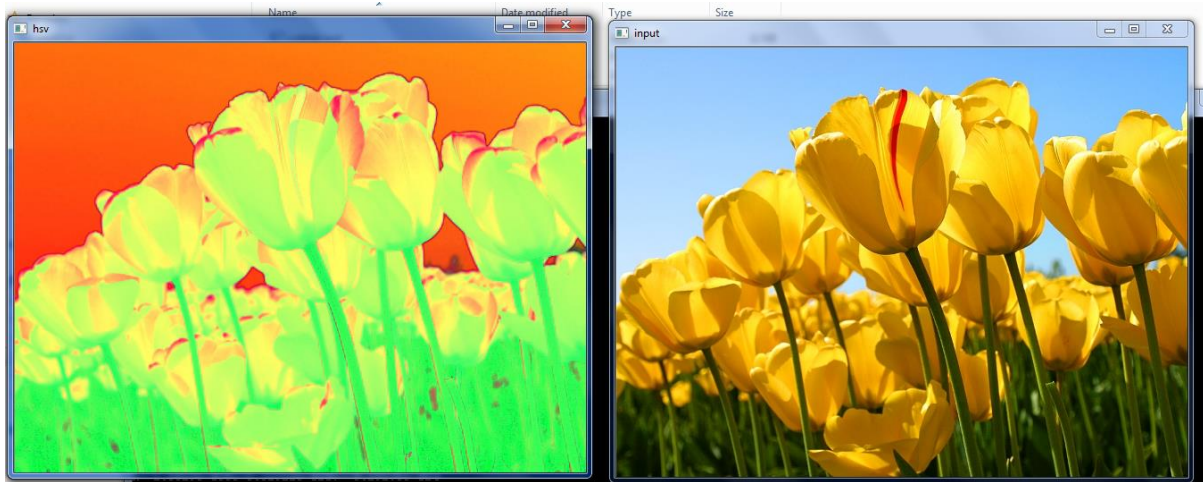


Applying Gaussian Blur: In image processing gaussian blur also known as gaussian smoothing is a result of blurring an image by gaussian function. It is widely used effect in graphics software typically used to reduce the image noise and image detail.

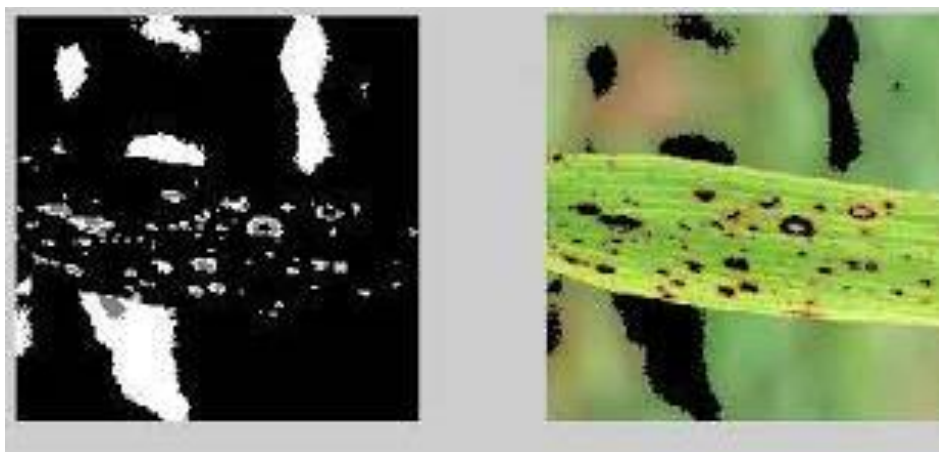


Converting RGB image into HSV image using green masking :

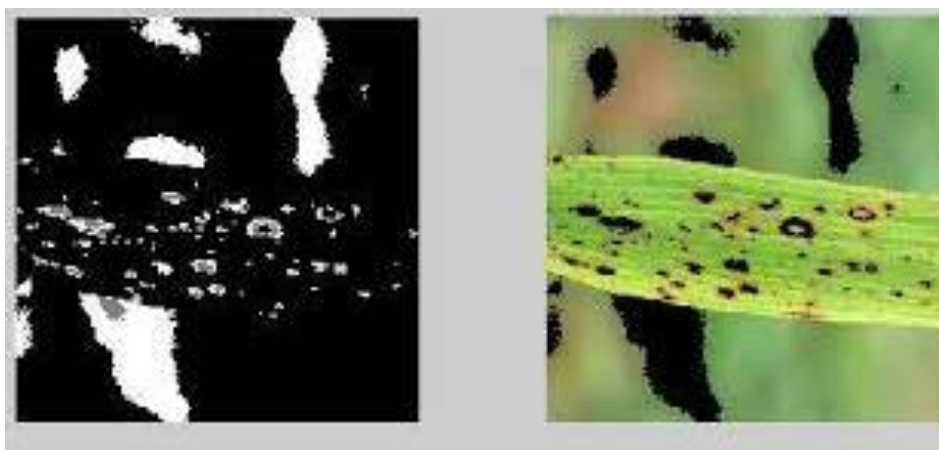
HSV Image: the term HSV stands for hue saturation value scale image provides a numerical readout of your image that corresponds to the color names contained there in. It is actually a type of color plane representation. The HSV color representation is useful to detect specific color types i.e; skin color, hue color etc....



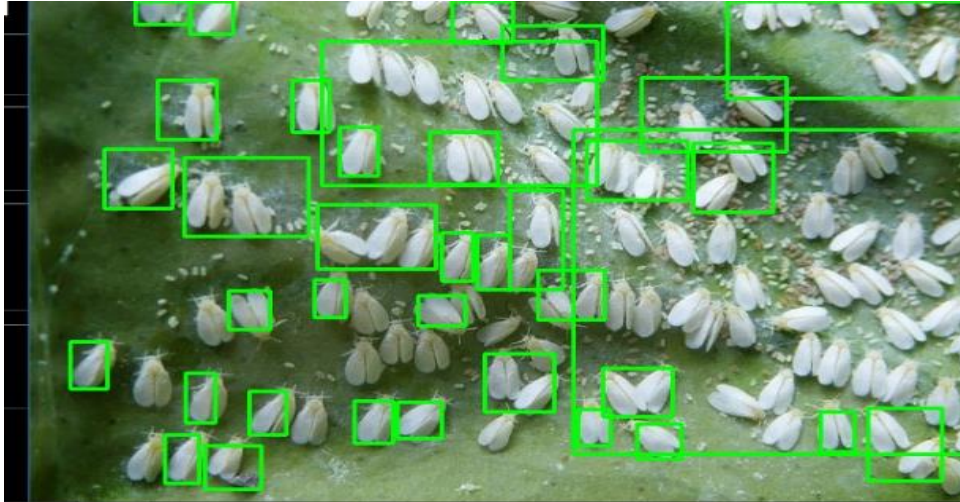
Green Masking : In our project the HSV image is used to detection of green color. Green masking is used to detect the colors in the HSV image and then make a note of the green colors that are closer to the lower and upper boundaries of the green color and convert the green pixels into white and the rest into black color.



Detection of pests : Once, the green masking technique is completed we get a binary image in combination of black and white pixels. Here, a complete white image denotes that there is no bug present in the system and the system represents the user that there is no bug in the image. If the output image is a binary image then this confirms the pest infestation in the field or in the image.



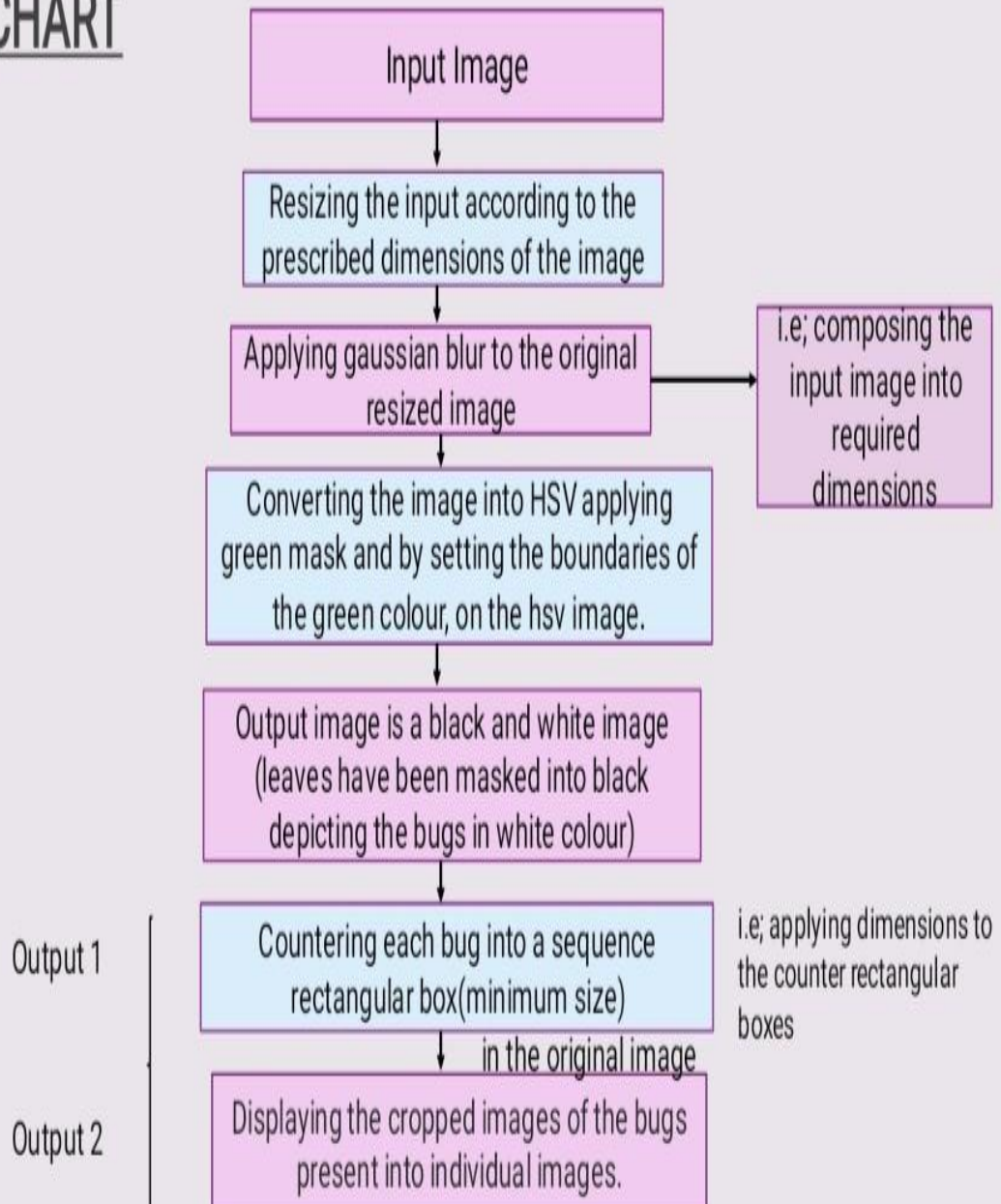
Countering the bugs in the original image : Once, after the confirmation or detection of pests in the image the pests present in the original images will be converted into rectangular boxes using the minimum rectangular area around the bugs. These counters shows the area of the image in which the bugs are present.



Displaying cropped images of the bugs : The countered bugs in the original input image will be cropped into individual images showing the bugs present clearly and specifically. Cropped images of the bugs helps in the easy viewing of different bugs present in the image.

Displaying bug count : Bug count refers to the number of bugs present in the image. Here, the bug count is same as the number of cropped images .i.e; the number of cropped images of the bugs depicts the no of bugs or count of the bugs present in the image in total.

FLOW CHART



5. CODING

The goal of the coding or programming phase is to translate the design of the system produced during the design phase into code in a given programming language, which can be executed by a computer and that performs the computation specified by the design.

The coding phase affects both testing and maintenance. The goal of coding is not to reduce the implementation cost but the goal should be to reduce the cost of later phases. In other words the goal is not to simplify the job of programmer. Rather the goal should be to simplify the job of the tester and maintainer.

5.1 Coding Approach:

There are two major approaches for coding any software system. They are Top-Down approach and bottom up approach.

Bottom-up Approach can best suit for developing the object-oriented systems. During system design phase of reduce the complexity. We decompose the system into an appropriate number of subsystems, for which objects can be modeled independently. These objects exhibit the way the subsystems perform their operations.

Once objects have been modeled they are implemented by means of coding. Even though related to the same system as the objects are implemented of each other the Bottom-Up approach is more suitable for coding these objects. In this approach, we first do the coding of objects independently and then we integrate these modules into one system to which they belong.

5.2 Information Handling:

Any software system require some amount of information during its operation selection of appropriate data structures can help us to produce the code so that objects of the system can better operate with the available information decreased complexity.

In the current system we followed the coding rules for naming the variables and methods. As part of coding internal documentation is also provided that help the readers to better understand the code.

5.3 Programming Style:

Programming style deals with act of rules that a programmer has to follow so that the characteristics of coding such as Traceability, Understandability, Modifiability, and Extensibility can be satisfied. In the current system we followed the coding rules for naming the variables and methods. As part of coding internal documentation is also provided that help the readers to better understand the code.

5.4 Verification and Validation:

Verification is the process of checking the product built is right. Validation is the process of checking whether the right product is built. During the Development of the system Coding for the object has been thoroughly verified from different aspects regarding their design, in the way they are integrated and etc. The various techniques that have been followed for validation discussed in testing the current system.

5.5 SAMPLE SOURCE CODE:

```
from tkinter import *
from PIL import ImageTk, Image
from tkinter.filedialog import asksaveasfile
from tkinter.filedialog import askopenfilename

import cv2
from PIL import Image
import numpy as np
import os
import imutils

def blur(image):
    blur = cv2.GaussianBlur(image, (11, 11), 0)

    return blur

def mask(image):
    greenLower = (27, 25, 25)
    greenUpper = (70, 255, 255)
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, greenLower, greenUpper)
    mask = cv2.erode(mask, None, iterations= 2)
    mask = cv2.dilate(mask, None, iterations= 2)
    mask = cv2.bitwise_not(mask)

    return mask

def contour(mask):

    cnts = cv2.findContours(mask.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    cnts = imutils.grab_contours(cnts)

    boxes = []

    for cnt in cnts:
        if cv2.contourArea(cnt) > 350:
            x, y, w, h = cv2.boundingRect(cnt)
            boxes.append([x, y, w, h])
```

```

    return boxes

def bounding(image,boxes):
    img = image.copy()
    for box in boxes:
        x,y,w,h = box
        cv2.rectangle(img,(x,y),(x+w,y+h),(0,255,0),2)

    return img

def crops(image,boxes):
    crop_list = []
    for box in boxes:
        x,y,w,h = box
        crop = image[y:y+h,x:x+w]
        crop = cv2.resize(crop,(240,360))
        crop_list.append(crop)

    return crop_list

def raise_frame(f):
    f.tkraise()

def finish():
    r.destroy()
def Fr1():
    raise_frame(f1)
def Fr2():
    raise_frame(f2)
def Fr3():
    raise_frame(f3)
def Fr4():
    raise_frame(f4)
def Fr5():
    raise_frame(f5)
def Fr6():
    raise_frame(f6)
def Fr7():
    raise_frame(f7)
def Fr8():
    raise_frame(f8)
def Fr9():
    raise_frame(f9)

```

```
#-----Page 1-----
-----
def fun1():

    global img
    global img_label

    path = askopenfilename(filetypes=[("Image File",'.jpg')])

    img_int = Image.open(path)
    img_int = img_int.resize((480,360),)
    img_pil = ImageTk.PhotoImage(img_int)
    img = np.asarray(img_int)
    img = cv2.cvtColor(img,cv2.COLOR_RGB2BGR)
    try:
        if img_label is not None:
            img_label.destroy()
    except:
        p = 3
    img_label = Label(f4,width=480,height=360,image=img_pil, anchor=CENTER)
    img_label.place(x= 250,y=30)
    img_label.pack()
    img_label.configure(image=img_pil)
    img_label.image()

def fun2():
    global blurs
    global img_pil

    blurs = blur(img)
    img_pil = ImageTk.PhotoImage(Image.fromarray(cv2.cvtColor(blurs,cv2.COLOR_RGB2BGR)))
    img_label.configure(image=img_pil)
    img_label.image(img_pil)

def fun3():
    global masks
    global img_pil
    masks = mask(blurs)
    img_pil = ImageTk.PhotoImage(Image.fromarray(masks))
    img_label.configure(image=img_pil)
    img_label.image(img_pil)

def fun4():
    global masks
    global final
    global img_pil
    global boxes

    boxes = contour(masks)
    final = bounding(img,boxes)
```

```

img_pil = ImageTk.PhotoImage(Image.fromarray(cv2.cvtColor(final,cv2.COLOR_RGB2BG
R)))
img_label.configure(image=img_pil)
img_label.image(img_pil)

def fun5():

    global c_list
    c_list = crops(img,boxes)
    for c in c_list:
        cv2.imshow('bugs',c)
        cv2.waitKey(0)
    cv2.destroyAllWindows()

def fun6():

    c_list = crops(img,boxes)
    num = len(c_list)
    count_var.set(str(num))

#-----Page 2-----
-----

def fun7():
    x7.set('')
def fun8():
    x8.set('')
def fun9():
    x9.set('')
def fun10():
    x10.set('')

r = Tk()
f0 = Frame(r)
f1 = Frame(r)
f2 = Frame(r)
f3 = Frame(r)
f4 = Frame(r)
f5 = Frame(r)
f6 = Frame(r)
f7 = Frame(r)
f8 = Frame(r)
f9 = Frame(r)

f0.place(x = 0,y = 0,height=800, width=1200)
f1.place(x = 100,y = 250,height=430, width=1000)
f2.place(x = 100,y = 250,height=430, width=1000)
f3.place(x = 100,y = 250,height=430, width=1000)
f4.place(x = 100,y = 250,height=430, width=1000)
f5.place(x = 100,y = 250,height=430, width=1000)
f6.place(x = 100,y = 250,height=430, width=1000)
f7.place(x = 100,y = 250,height=430, width=1000)
f8.place(x = 100,y = 250,height=430, width=1000)
f9.place(x = 100,y = 250,height=430, width=1000)

```

```

x0=StringVar()
x1=StringVar()
x2=StringVar()
x3=StringVar()
x4=StringVar()
x5=StringVar()
x6=StringVar()
x7=StringVar()
x8=StringVar()
x9=StringVar()
x10=StringVar()
x11=StringVar()
x12=StringVar()
x13=StringVar()
x14=StringVar()
x15=StringVar()
x16=StringVar()
sc=StringVar()
sk=StringVar()
oc=StringVar()
ok=StringVar()

f0.configure(bg="black")
f1.configure(bg="black")
f2.configure(bg="black")
f3.configure(bg="black")
f4.configure(bg="black")
f5.configure(bg="black")
f6.configure(bg="black")
f7.configure(bg="black")
f8.configure(bg="black")
f9.configure(bg="black")

#HeadingPage
Label(f0, text = "",bg='gray',height=9,width=165).place(x = 20, y = 20)

Label(f0, justify='center',bg='gray',text = ""GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND P.G. COURSES(A) RUSHIKONDA, VISAKHAPATNAM-45 DEPARTMENT OF COMPUTER APPLICATION"" ,fg='white',height=4,font = "Helvetica 18 bold").place(x = 200, y = 30)

ph2=ImageTk.PhotoImage(Image.open("logo1.jpg"))
Label(f0, text = "",image=ph2,height=0).place(x = 50, y = 30)

b2 = Button(f0, text = "Home",fg="white",bg="black", font = "Helvetica 13 bold",height=2,width=10,command = Fr1).place(x = 227, y =180)
b3 = Button(f0, text = "Abstract",fg="white",bg="black", font = "Helvetica 13 bold",height=2,width=10,command = Fr2).place(x = 354, y =180)
b4 = Button(f0, text = "Proposed",fg="white",bg="black", font = "Helvetica 13 bold",height=2,width=10,command = Fr3).place(x = 481, y =180)
b5 = Button(f0, text = "Methodology",fg="white",bg="black", font = "Helvetica 13 bold",height=2,width=10,command = Fr4).place(x = 608, y =180)

```

```

b6 = Button(f0, text = "Conclusion",fg="white",bg="black", font = "Helvetica 13 bold",height=2,width=10,command = Fr8).place(x = 735, y =180)
b7 = Button(f0, text = "ThanQ",fg="white",bg="black", font = "Helvetica 13 bold",height=2,width=10,command = Fr9).place(x = 862, y =180)

#-----StartPage---Frame 1-----
-----

Label(f1, text = ""Pest Detection and Extraction using Image Processing Techniques"",fg="white",bg='black',font = "Georgia 40 bold",height=2).place(x = 20, y = 50)

Label(f1, text = ""Project by
      K.Poojitha (2018-1902048)
      N.Pavan Choudhury (2018-1902060)
      Aasim Ali shaik (2018-1902055)
      BCA VI SEMESTER
      "",fg="purple",bg="black",font = "Georgia 14 bold",height=6).place(x = 20, y = 250)

Label(f1, text = ""Project Guide
Mrs. ch Suneetha
Assistant Professor
      "",fg="white",bg='black',font = "Georgia 14 bold",height=6,width=20).place(x = 600, y = 250)

#-----Frame-2-----
-----

Label(f2, text = "Abstract ",fg="white",bg="black", font = "Latin 18 bold").place(x = 50,y=30)

Label(f2,justify="left", text=""
Detection of pests in the fields is a major challenge in the field of agriculture, therefore effective measures should

be developed to fight the infestation while minimizing the use of pesticides. The techniques of image analysis are

extensively applied to agricultural science and it provides maximum protection to crops which can ultimately lead

to better crop management and production. Monitoring of pests infestation requires a huge amount of manpower,

however automatic monitoring has been advancing in order to minimize human efforts and errors. This project extends

the implementation of different image processing techniques to detect and extract insect pests by establishing an

automated detection and extraction system for estimating pest densities in paddy fields.""",fg="white",bg="black", font = "Latin 13 bold").place(x = 50, y =80)

```

```

b7 = Button(f2, text = "Prev",fg="white",bg="black", font = "Helvetica 14 bold",height=1,width=6,command=Fr1).place(x = 10, y = 385)
b8 = Button(f2, text = "Next",fg="white",bg="black", font = "Helvetica 14 bold",height=1,width=6,command = Fr3).place(x = 920, y = 385)

#-----Frame 3-----
-----

l1 = Label(f3, text = "Proposed System",fg="white",bg="black", font = "Latin 18 bold").place(x = 50,y=30)
l1 = Label(f3, text = ""
Image processing : Image processing is the analysis and manipulation of graphical images from sources such as photographs and videos

There are three main steps in image processing :

    considering a captured image, converting the captured images into binary values that a computer can process.

    Image enhancement and data compression.

    This is an output step that consists of display and printing of processed image.""",fg="white",bg="black",justify="left", font = "Latin 13 bold").place(x = 50,y=80)

b7 = Button(f3, text = "Prev",fg="white",bg="black", font = "Helvetica 14 bold",height=1,width=6,command=Fr2).place(x = 10, y = 385)
b8 = Button(f3, text = "Next",fg="white",bg="black", font = "Helvetica 14 bold",height=1,width=6,command = Fr4).place(x = 920, y = 385)

#-----Frame-4-----
-----

L0= Label(f4,text="Methodology",fg="black",bg="white",font="Latin 18 bold").place(x=10,y=10)

#img_label = Label(f4,fg="black",bg="light gray", font = "Latin 12 bold",width=50,height=15).place(x = 250,y=60)

B1= Button(f4,text="Input Image",fg="white",bg="black",font="Latin 12 bold",width=16,height=2,command=fun1).place(x=20,y=60)

B1= Button(f4,text=""Gaussian
    Blur"",fg="white",bg="black",font="Latin 12 bold",width=16,height=3,command=fun2).place(x=20,y=120)
#l3= Label(f4,textvariable=x2,fg="black",bg="light gray", font = "Latin 12 bold",width=50,height=2).place(x = 250,y=120)

B2= Button(f4,text=""HSV Image
using Green
masking
"",fg="white",bg="black",font="Latin 12 bold",width=16,height=4,command=fun3).place(x=20,y=200)

B3= Button(f4,text=""Pest

```

Detection

```
""",fg="white",bg="black",font="Latin 12 bold",width=16,height=3,command=fun4).place(x=20,y=300)
```

```
#L6= Label(f4,textvariable=x4,fg="black",bg="light gray",font="Latin 12 bold",width=50,height=2).place(x=250,y=240)
```

```
b1 = Button(f4, text = ""Cropped bug
```

```
Images""",fg="white",bg="black", font = "Latin 12 bold",height=2,width=16,command = fun5).place(x =830, y =60)
```

```
#l2 = Label(f4, textvariable=x6,fg="black",bg="light gray", font = "Latin 12 bold",width=50,height=15).place(x = 250, y =60)
```

```
count_var = StringVar()
```

```
count_var.set('0')
```

```
l5 = Label(f4,textvariable=count_var,fg="black",bg="white",font="Latin 18 bold")
```

```
l5.place(x=830,y=180)
```

```
b2 = Button(f4, text = "Bug Count",fg="white",bg="black", font = "Latin 12 bold",height=2,width=16,command = fun6).place(x = 830, y =120)
```

```
b7 = Button(f4, text = "Prev",fg="white",bg="black", font = "Helvetica 14 bold",height=1,width=6,command=Fr3).place(x = 10, y = 385)
```

```
b8 = Button(f4, text = "Next",fg="white",bg="black", font = "Helvetica 14 bold",height=1,width=6,command = Fr8).place(x = 920, y = 385)
```

```
#-----Frame-5-----  
-----
```

```
l1 = Label(f8, text = "CONCLUSION ",fg="white",bg="black", font = "Latin 16 bold").place(x = 30,y=10)
```

```
l1 = Label(f8,  
text = ""
```

In this system, the automatic detection and extraction system was presented, different image processing

techniques were used to detect and extract the pests in the captured image. The presented problem statement is

simple and yet efficient. We used background modelling to detect the presence of insect pests in the captured

image and a median filter was used to remove the noise produced by different light conditions. The mechanism

used to extract the detected objects from the image is simple, the image was scanned both horizontally and

vertically to determine each coordinates and save the object image. The problem statement is promising, efficient

and simple in providing better results which in turn results in the improvement of quality and quantity of crop in

```

the rice fields. """,fg="white",bg="black",justify='left', font = "Latin 13 bold").p
lace(x = 30,y=50)

b7 = Button(f8, text = "Prev",fg="white",bg="black", font = "Helvetica 14 bold",heig
ht=1,width=6,command=Fr4).place(x = 10, y = 385)
b8 = Button(f8, text = "Next",fg="white",bg="black", font = "Helvetica 14 bold",heig
ht=1,width=6,command = Fr9).place(x = 920, y = 385)

#-----Frame-6-----
-----

ph22=ImageTk.PhotoImage(Image.open("t.jpg"))
lab1 = Label(f9, text = "",image=ph22,height=0).place(x = -20, y = 0)

b7 = Button(f9, text = "Prev",fg="white",bg="black", font = "Helvetica 14 bold",heig
ht=1,width=6,command=Fr8).place(x = 10, y = 385)
b8 = Button(f9, text = "Next",fg="white",bg="black", font = "Helvetica 14 bold",heig
ht=1,width=6,command = Fr1).place(x = 920, y = 385)

#-----End-----
-----

raise_frame(f1)
r.geometry("1200x700+70+0")
r.title("Pest Detection and Extraction using Image Processing Techniques")
r.mainloop()

```

6. TESTING

Testing is the process of finding differences between the expected behaviour specified by system models and the observed behaviour of the system. Testing is a critical role in quality assurance and ensuring the reliability of development and these errors will be reflected in the code so the application should be thoroughly tested and validated.

Testing is the process of finding differences between the expected behaviour specified by system models and the observed behaviour of the system. Testing is a critical role in quality assurance and ensuring the reliability of development and these errors will be reflected in the code so the application should be thoroughly tested and validated.

Unit testing finds the differences between the object design model and its corresponding components. Structural testing finds differences between the system design model and a subset of integrated subsystems. Functional testing finds differences between the use case model and the system.

Finally performance testing, finds differences between non-functional requirements and actual system performance. From modeling point of view, testing is the attempt of falsification of the system with respect to the system models. The goal of testing is to design tests that exercise defects in the system and to reveal problems.

6.1 Different levels of testing:

Unit Testing:

In this test each component is tested independent of the other thus allowing parallelism in testing activity.

Integration Testing:

Many tested modules are combined into subsystem. The emphasis is on testing interface between modules. This activity can be considered to test the design.

System Testing:

This is a series of testing whose purpose is to fully exercise the entire software system. System testing verifies that all elements mesh properly and the overall system function/performance is achieved. System testing is done using real data. The system performance was found to be working to be as per requirements specified.

System Testing:

- System testing ensures that the complete system compiles with the functional requirements and non-functional requirements of the system, the following are some system testing activities.
- Functional testing finds differences between the functional between the functional requirements and the system. This is a black box testing technique. Test cases are divided from the use case model.
- Performance testing finds differences between the design and the system the design goals are derived from the functional requirements.
- Pilot testing the system is installed and used by a selected set of users – users exercise the system as if it had been permanently installed.
- Acceptance testing, I have followed benchmarks testing in a benchmarks testing the client prepares a set of test cases represent typical conditions under which the system operates. In our project, there are no existing benchmarks.

But, there are certain benchmarks available on the Internet regarding the minimum and maximum time limits for generating the random Big Integer values. We have used these to test the system. Installation testing, the system is installed in the target environment.

6.2 Testing Approaches:

There are three basic approaches to testing:

- Functional (Black box testing)
- Conditional
- Structural (White box testing)

White box Testing:

This test focuses on the program control structure. Test cases are derived to ensure that all the statements in the program have been executed at least once during testing and that all logical conditions have been exercised.

Condition testing:

Condition testing is a test case design method that exercises the logical conditions contained in a module the condition testing focuses on testing each condition in the operation.

Black box Testing:

These tests are designed to validate functional requirements without regard to working of the program. Black box testing technique focuses on the information domain of the software deriving test cases by partitioning input and output in a manner that provides thorough test coverage. This

approach also deals with the identification of class libraries or software packages or other application systems, which are already present so that developer can implement the object models with a decreased burden of testing.

6.3 Testing Activities:

Testing a large system is a complex activity and like any complex activity. It has to be broke into smaller activities. Thus incremental testing was performed on the project i.e., components and subsystems of the system were tested separately before integrating them to form the subsystem for system testing.

6.4 Testing Plan:

Testing accounts for 45 - 75% of the typical project effort. It is also one of the most commonly underestimated activities on a project. A test plan is a document that answers the basic questions about your testing effort. It needs to be initiated during the requirements gathering phase of your project and should evolve into a roadmap for the testing phase.

- Test Planning enables a more reliable estimate of the testing effort up front.
- It allows the project team time to consider ways to reduce the testing effort without being under time pressure.
- Test Plan helps to identify problem areas and focuses the testing team's attention on the critical paths.
- Test plan reduces the probability of implementing non-tested components.

7. RESULT

HOME



GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND P.G. COURSES(A)
RUSHIKONDA, VISAKHAPATNAM-45
DEPARTMENT OF COMPUTER APPLICATION

[Home](#) [Abstract](#) [Proposed](#) [Methodology](#) [Conclusion](#) [ThanQ](#)

Pest Detection and Extraction using Image Processing Techniques

Project by
K.Poojitha (2018-1902048)
N.Pavan Choudhury (2018-1902060)
Aasim Ali shaik (2018-1902055)
BCA VI SEMESTER

Project Guide
Mrs. ch Suneetha
Assistant Professor

ABSTRACT



GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND P.G. COURSES(A)
RUSHIKONDA, VISAKHAPATNAM-45
DEPARTMENT OF COMPUTER APPLICATION

[Home](#) [Abstract](#) [Proposed](#) [Methodology](#) [Conclusion](#) [ThanQ](#)

Abstract

Detection of pests in the fields is a major challenge in the field of agriculture, therefore effective measures should be developed to fight the infestation while minimizing the use of pesticides. The techniques of image analysis are extensively applied to agricultural science and it provides maximum protection to crops which can ultimately lead to better crop management and production. Monitoring of pests infestation requires a huge amount of manpower, however automatic monitoring has been advancing in order to minimize human efforts and errors. This project extends the implementation of different image processing techniques to detect and extract insect pests by establishing an automated detection and extraction system for estimating pest densities in paddy fields.

[Prev](#) [Next](#)

PROPOSEDSYSTEM



GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND P.G. COURSES(A)
RUSHIKONDA, VISAKHAPATNAM-45
DEPARTMENT OF COMPUTER APPLICATION

[Home](#)[Abstract](#)[Proposed](#)[Methodology](#)[Conclusion](#)[ThanQ](#)

Proposed System

Image processing : Image processing is the analysis and manipulation of graphical images from sources such as photogr

There are three main steps in image processing :

considering a captured image, converting the captured images into binary values that a computer can process.

Image enhancement and data compression.

This is an output step that consists of display and printing of processed image.

[Prev](#)[Next](#)

METHODOLOGY



GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND P.G. COURSES(A)
RUSHIKONDA, VISAKHAPATNAM-45
DEPARTMENT OF COMPUTER APPLICATION

[Home](#)[Abstract](#)[Proposed](#)[Methodology](#)[Conclusion](#)[ThanQ](#)

Methodology

Input Image

Gaussian Blur

HSV Image using Green masking

Pest Detection

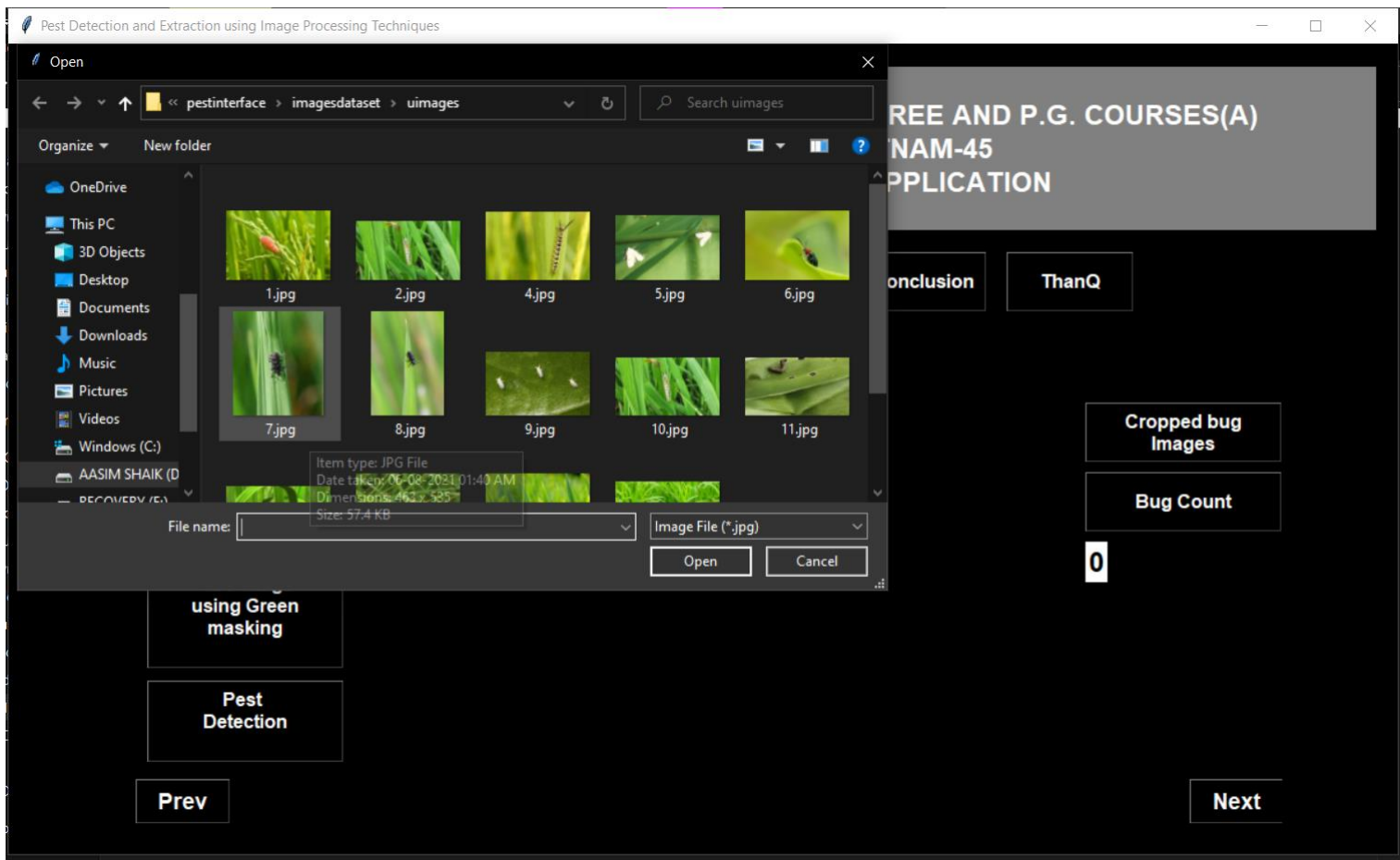
Cropped bug Images

Bug Count

0

[Prev](#)[Next](#)

INPUTIMAGE



INPUTIMAGE1



GAUSSIANBLUR



GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND P.G. COURSES(A)
RUSHIKONDA, VISAKHAPATNAM-45
DEPARTMENT OF COMPUTER APPLICATION

[Home](#)[Abstract](#)[Proposed](#)[Methodology](#)[Conclusion](#)[ThanQ](#)

Methodology

Input Image

Gaussian Blur

HSV Image using Green masking

Pest Detection



Cropped bug Images

Bug Count

0

Prev

Next

HSVGREENMASKING



GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND P.G. COURSES(A)
RUSHIKONDA, VISAKHAPATNAM-45
DEPARTMENT OF COMPUTER APPLICATION

[Home](#)[Abstract](#)[Proposed](#)[Methodology](#)[Conclusion](#)[ThanQ](#)

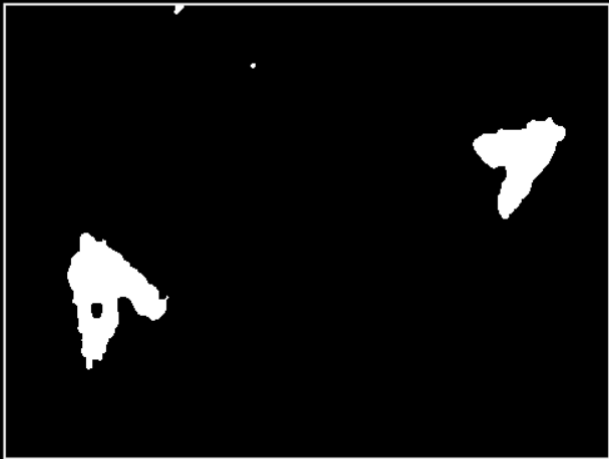
Methodology

Input Image

Gaussian Blur

HSV Image using Green masking

Pest Detection



Cropped bug Images

Bug Count

0

Prev

Next

PESTDetecting AND CONTOURING

Pest Detection and Extraction using Image Processing Techniques



GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND P.G. COURSES(A)
RUSHIKONDA, VISAKHAPATNAM-45
DEPARTMENT OF COMPUTER APPLICATION

[Home](#) [Abstract](#) [Proposed](#) [Methodology](#) [Conclusion](#) [ThanQ](#)

Methodology

[Input Image](#)
[Gaussian Blur](#)
[HSV Image using Green masking](#)
[Pest Detection](#)



[Cropped bug Images](#)
[Bug Count](#)
0

[Prev](#) [Next](#)

CROPPINBUGS

Pest Detection and Extraction using Image Processing Techniques

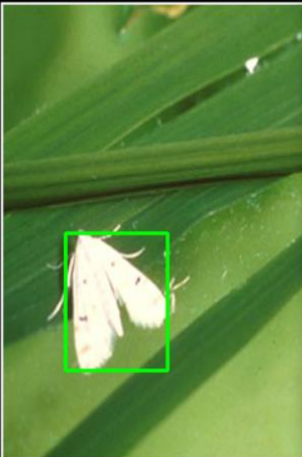


GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND P.G. COURSES(A)
RUSHIKONDA, VISAKHAPATNAM-45
DEPARTMENT OF COMPUTER APPLICATION

[Home](#) [Abstract](#) [Proposed](#) [Methodology](#) [Conclusion](#) [ThanQ](#)

Methodology

[Input Image](#)
[Gaussian Blur](#)
[HSV Image using Green masking](#)
[Pest Detection](#)

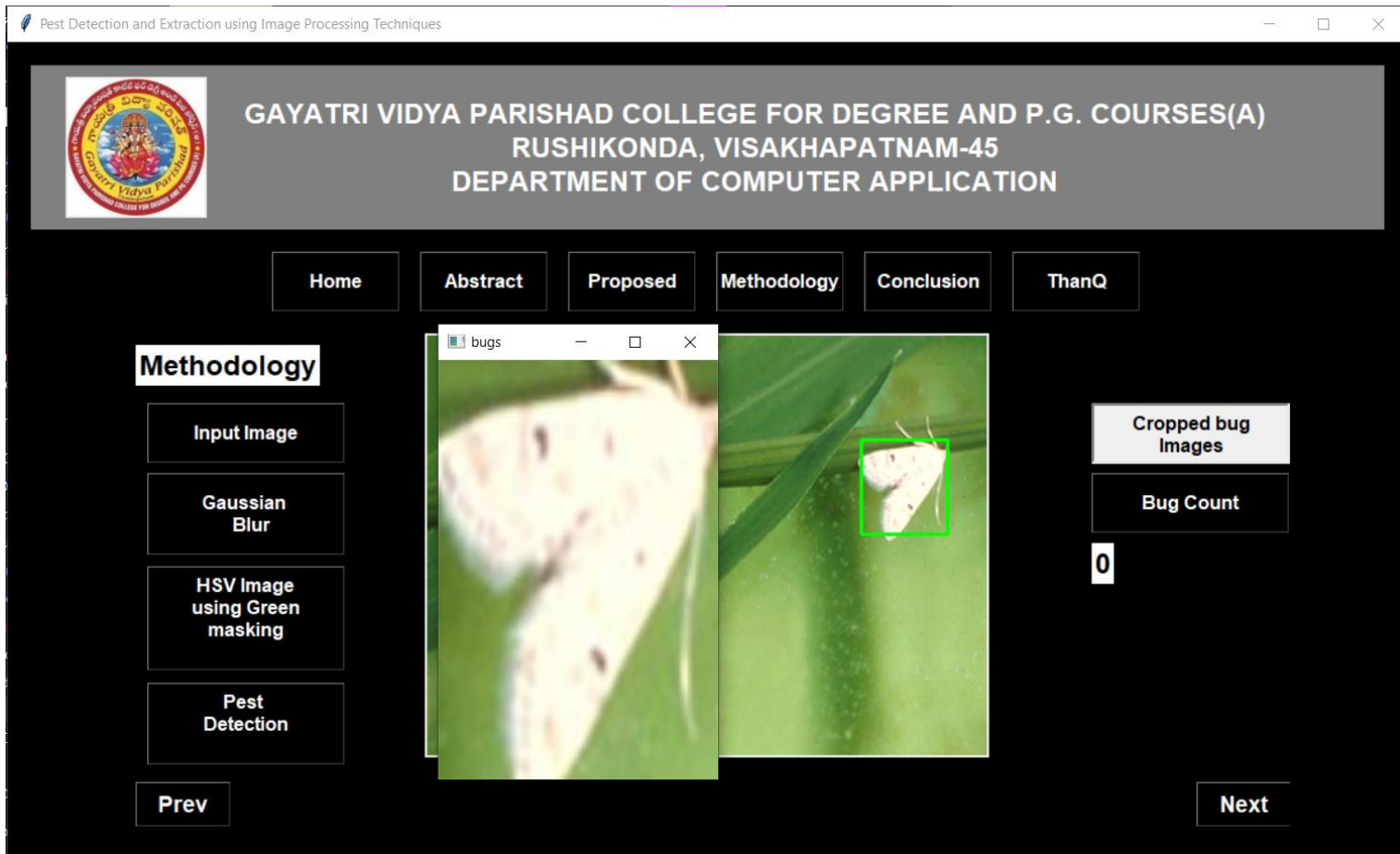




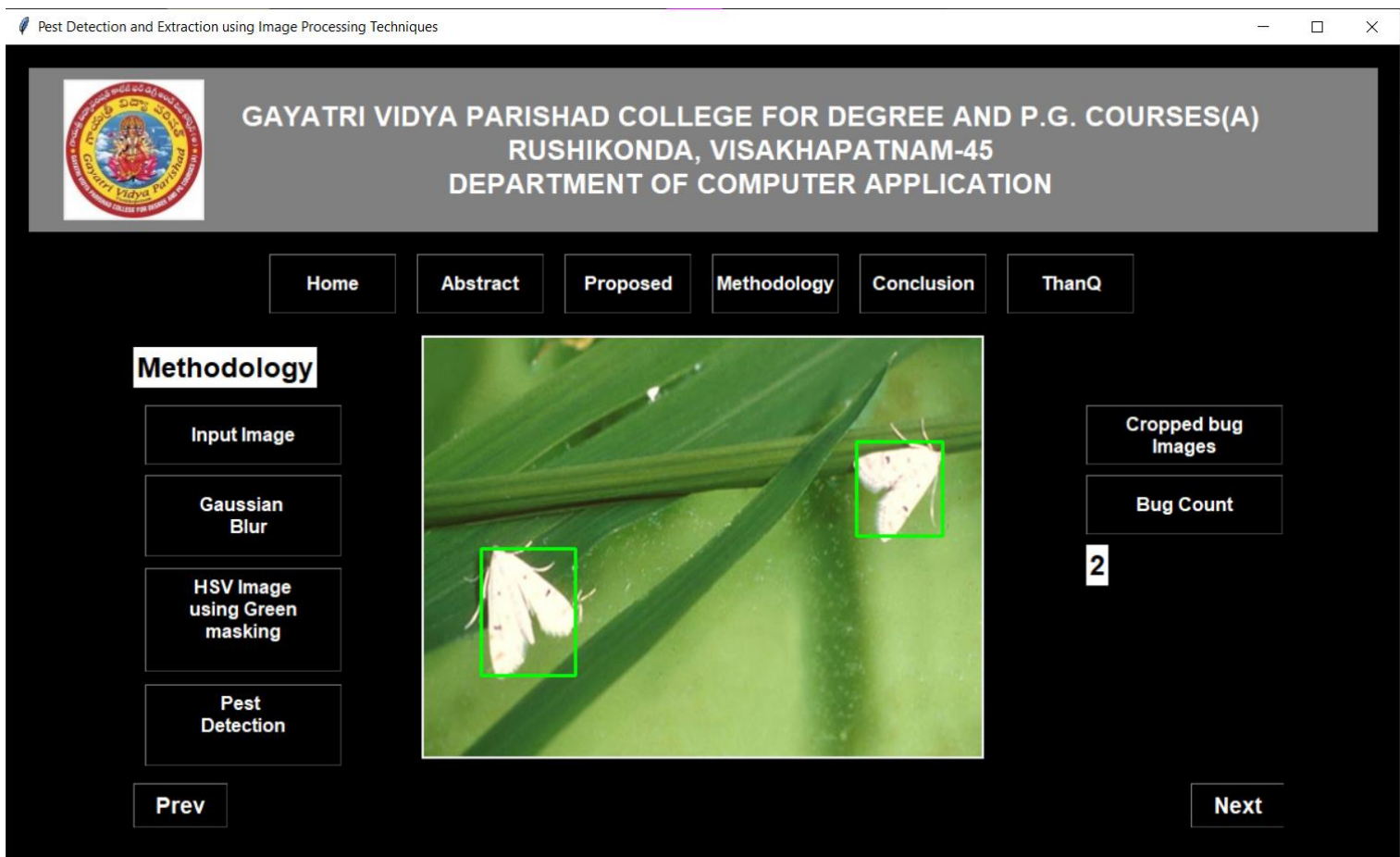
[Cropped bug Images](#)
[Bug Count](#)
0

[Prev](#) [Next](#)

CROPPINGBUGS



BUG COUNT



CONCLUSION



**GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND P.G. COURSES(A)
RUSHIKONDA, VISAKHAPATNAM-45
DEPARTMENT OF COMPUTER APPLICATION**

[Home](#) [Abstract](#) [Proposed](#) [Methodology](#) [Conclusion](#) [ThanQ](#)

CONCLUSION

In this system, the automatic detection and extraction system was presented, different image processing techniques were used to detect and extract the pests in the captured image. The presented problem statement is simple and yet efficient. We used background modelling to detect the presence of insect pests in the captured image and a median filter was used to remove the noise produced by different light conditions. The mechanism used to extract the detected objects from the image is simple, the image was scanned both horizontally and vertically to determine each coordinates and save the object image. The problem statement is promising, efficient and simple in providing better results which in turn results in the improvement of quality and quantity of crop in the rice fields.

[Prev](#) [Next](#)

END



**GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND P.G. COURSES(A)
RUSHIKONDA, VISAKHAPATNAM-45
DEPARTMENT OF COMPUTER APPLICATION**

[Home](#) [Abstract](#) [Proposed](#) [Methodology](#) [Conclusion](#) [ThanQ](#)



[Prev](#) [Next](#)

8. CONCLUSION

In this system, the automatic detection and extraction system was presented, different image processing techniques were used to detect and extract the pests in the captured image. The presented problem statement is simple and yet efficient.

We used background modelling to detect the presence of insect pests in the captured image and a median filter was used to remove the noise produced by different light conditions. The mechanism used to extract the detected objects from the image is simple, the image was scanned both horizontally and vertically to determine each coordinates and save the object image.

The problem statement is promising, efficient and simple in providing better results which in turn results in the improvement of quality and quantity of crop in the rice fields.

10.APPENDIX

10.1 List of Figures:

Figure no.	Name of the Figure	Page no
1.1		10
1.2		11

1.3		12
1.4		14
1.5		15
2.6		18
3.1.1		19
3.1.2	Usecase Diagram	21
3.2.1	Sequence Diagram	22
3.3.1	State Diagram	23
3.3.2		24
3.4		25
4.2.2		29
7.1		69
7.2		69
7.3		70
7.4		70
7.5		71
7.6		71
7.7		72
7.8		72
7.9		73
7.10		73